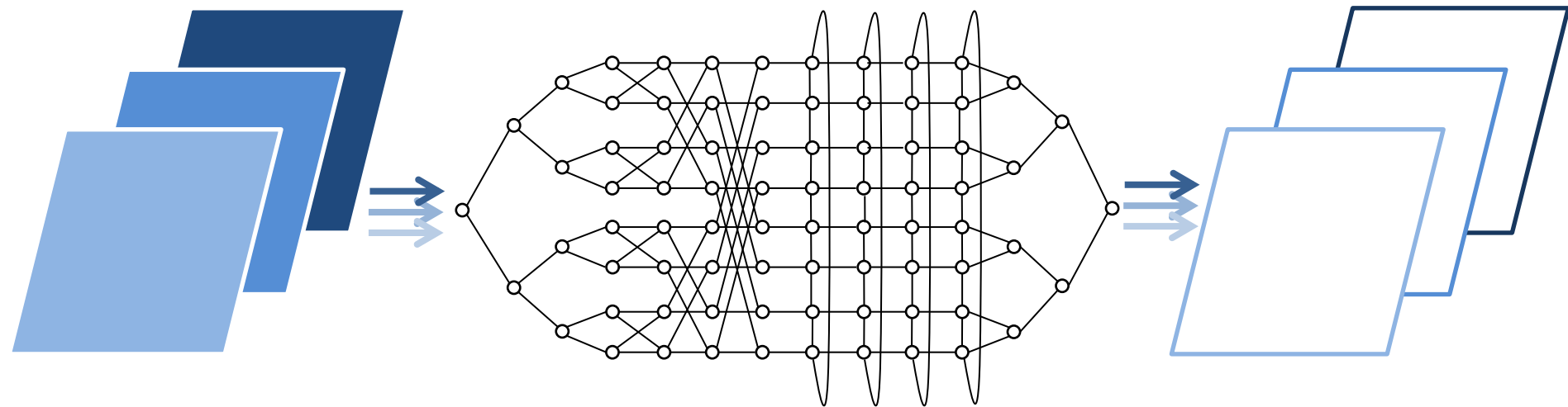


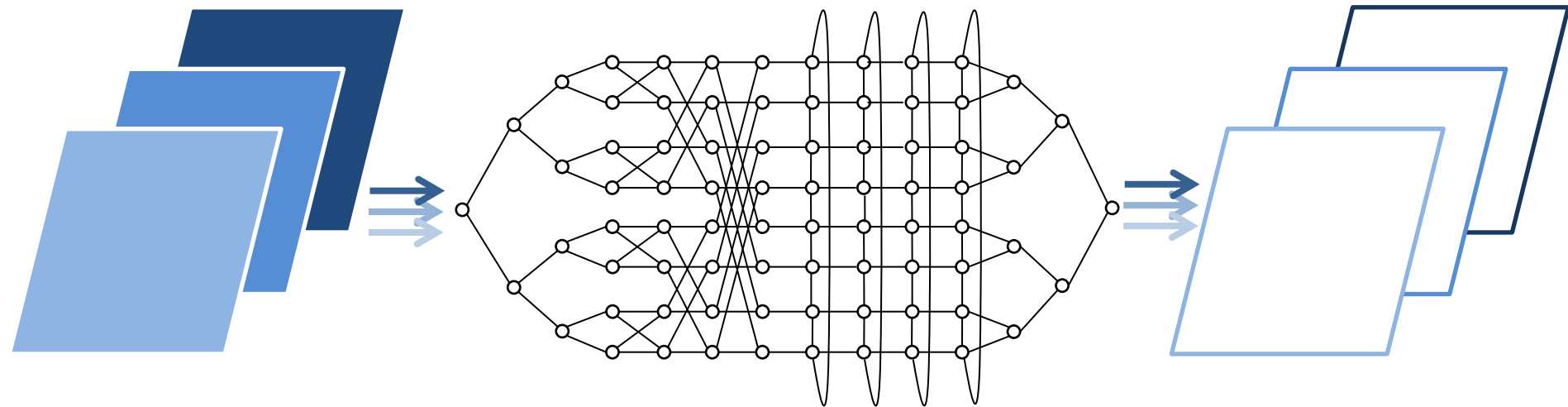
Programmation Parallèle

MPI: Message Passing Interface



Introduction to HPC and Parallel Programming

ENSIIE-HPC/BigData-PP-IIP-Lecture 0



Context



- Why do we need parallel computing?
- How can we exploit main resources from computers?
- What are the different approaches/paradigms of parallelism?

Context



■ Why parallelism?

- Parallel computing is everywhere
- From cellphone processors to supercomputers

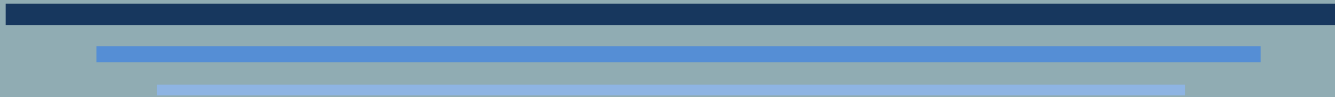
■ Parallel programming paradigm

- Distributed-memory programming
- Shared-memory programming
- Hybrid programming
- Heterogeneous programming

■ Goal

- Identify independent work to exploit parallelism

SUPERCOMPUTER ARCHITECTURE

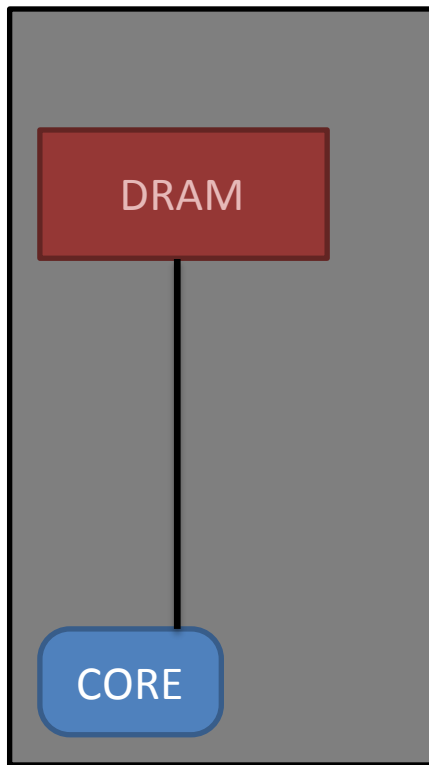


MPI: Message Passing Interface

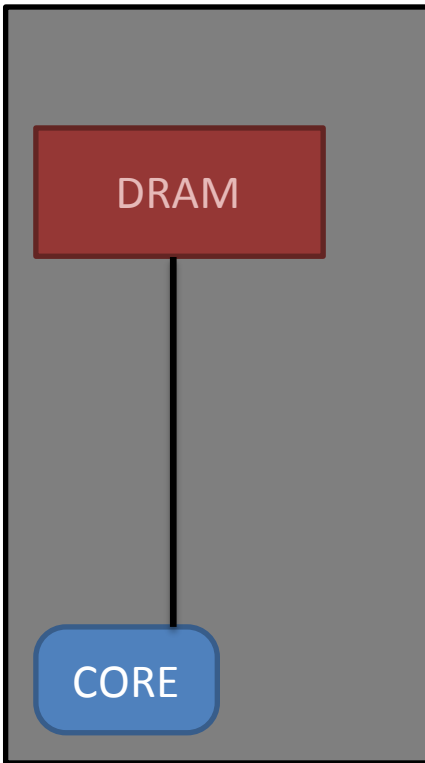
Once upon a time, ...



- There was an Arithmetic and Logical Unit (ALU) able to realize operations:
 - Arithmetic (add, sub, mul, ...)
 - Logical (AND, OR, ...)
- ...and a memory to store data

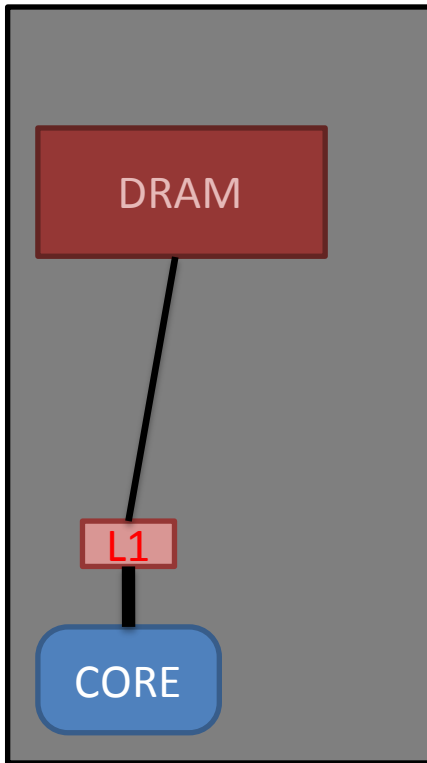


Hitting the memory wall



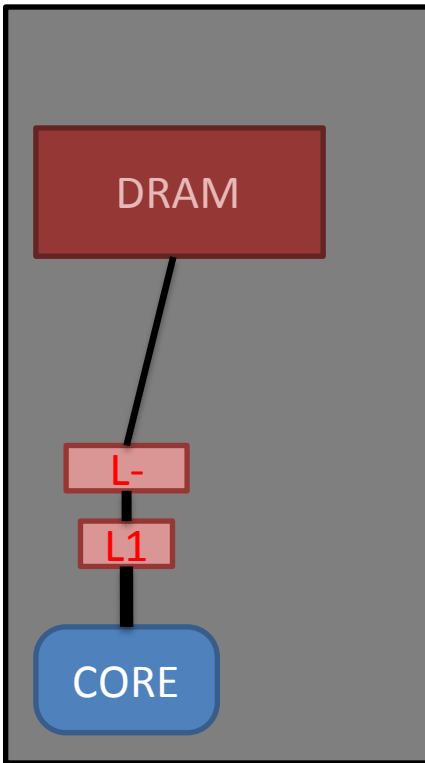
- Performance gain was achieved with increase of compute power
 - frequency
- Problem: compute performances increase faster than memory performances
- Need to find ways to “feed” the compute unit
 - Otherwise, it is not possible to achieve peak performance

Let there be caches



- So a data cache was added between compute unit and data memory
- Higher performance memory closer to the compute unit
 - Better latency
 - Better bandwidth
- Act like a “buffer” memory close to the compute unit
 - Keep temporarily last used data

HPC saw it was good!

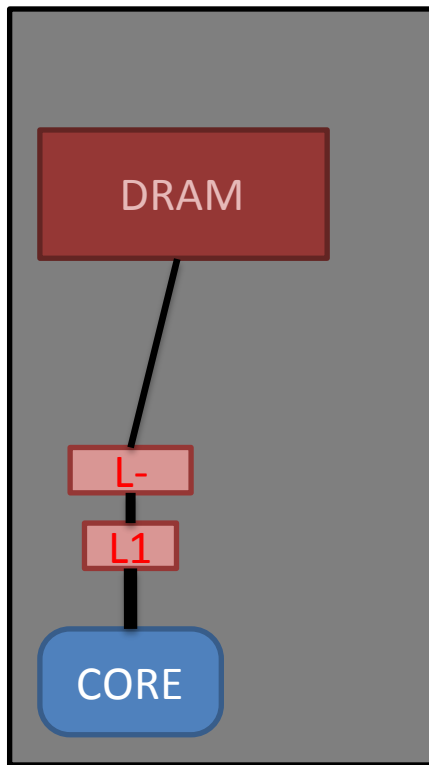


- Only one level of cache may not be enough
 - Too small to keep all data of interests
 - -> hard to play its role and feed the compute unit efficiently
- Add more cache levels
 - Depending on architecture (often 2 or 3 levels)
 - Further the cache, lower the bandwidth and higher latency and capacity

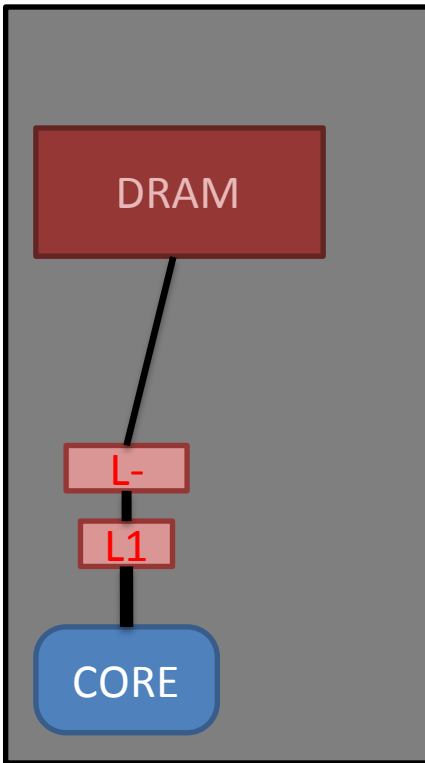
Scotty, we need more (compute) power



- Each new generation of supercomputers (and processor) generation aims at providing more compute power
- Main way to reach more compute power: increase the processor frequency

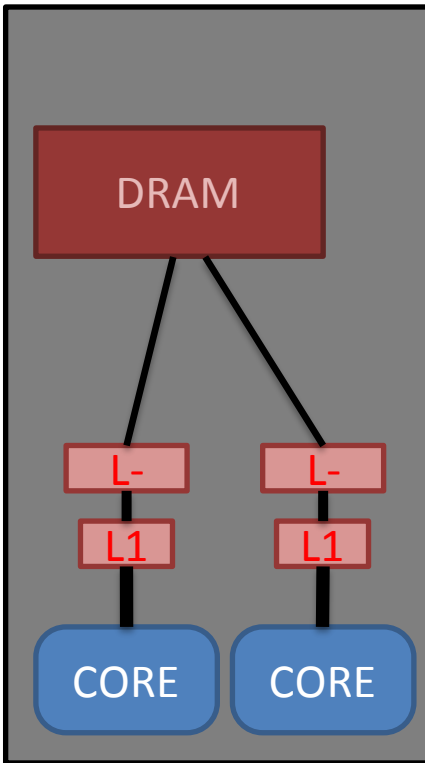


Hitting the heating wall



- Increase frequency is done with increase of number of transistors
- Usually roughly keeping the same size of chip
 - Leads to reducing transistor size
- Density of transistor keeps increasing
 - So does the electrical generated heat
 - Double the density, n^2 increase of heat
 - Too much heat impossible to diffuse

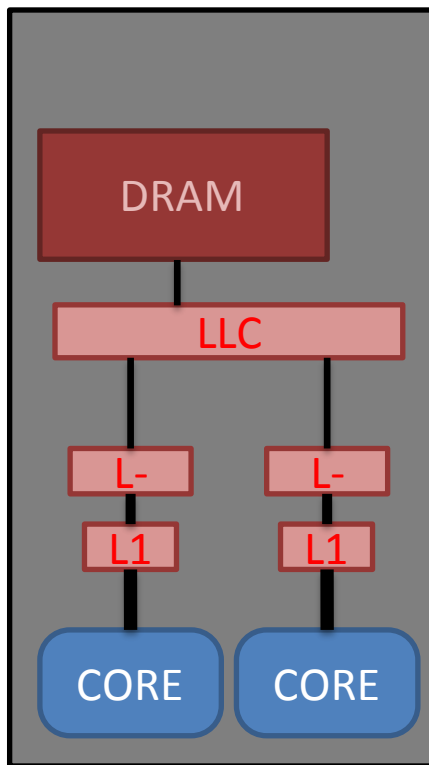
Multiplying cores



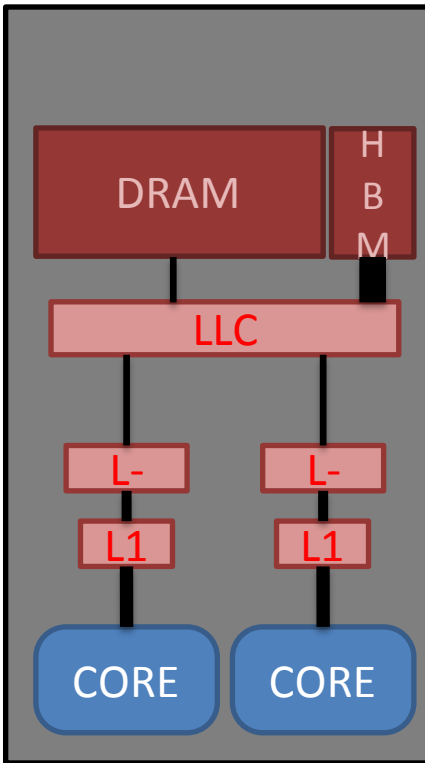
- So, instead of increasing number of transistor per compute unit...
- ... let's multiply number of compute units
- Allows to increase compute power while limiting proximity and density of transistors
 - Double the cores, double the heat

Caches are gooooooood!

- Sharing data between cores can happen only through global memory
 - Not very efficient if the cores need to use the same data
 - In this case, caches are not useful
- Often, the last level cache (LLC) is shared between cores
 - Avoid to go the main memory each time a core need a shared data
 - Also possible to have intermediate shared caches

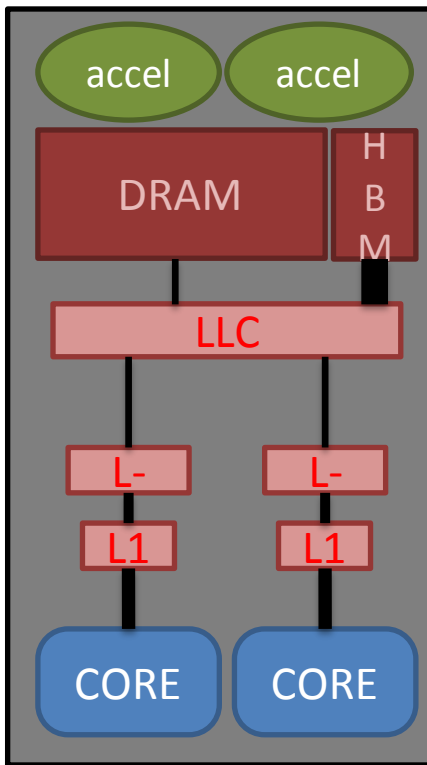


Always need more data



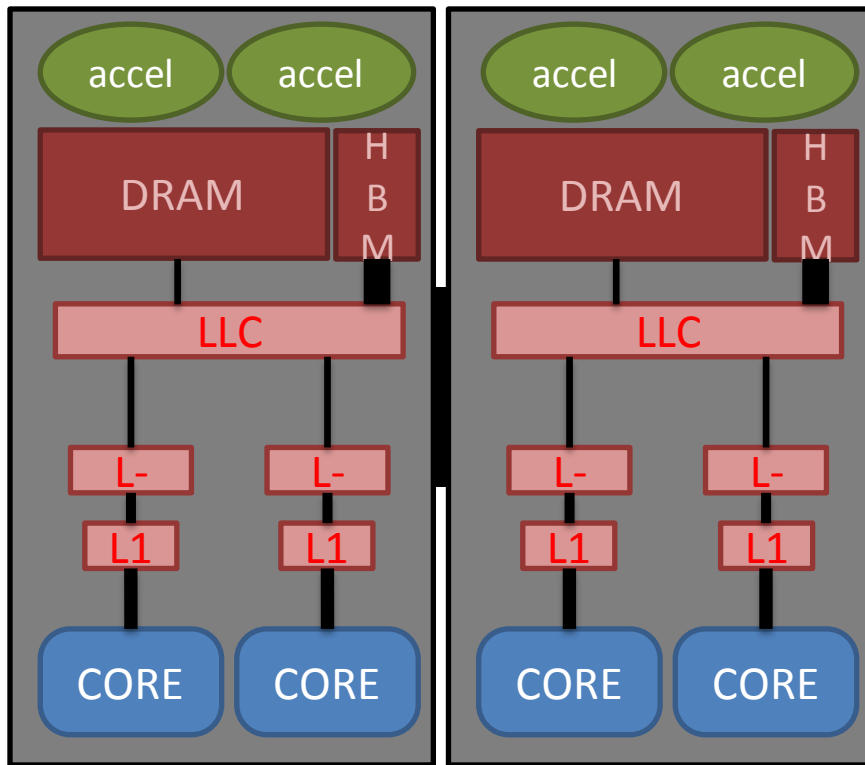
- Increasing number of cores (and generally compute power) requires always more data to be transferred
- Even with caches, having only one memory with low bandwidth becomes problematic
- Adding more global memories, with different characteristics (HBM, NVMe)

Always need more compute power (with capped power)



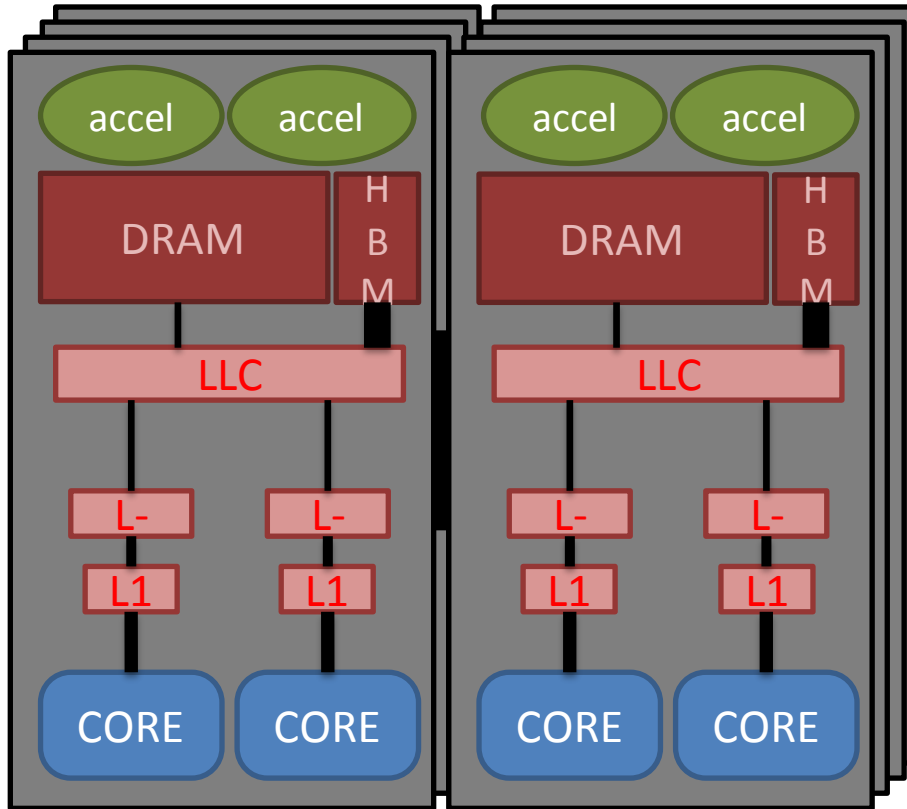
- Cores are built to handle every possible operations
 - Require lots of transistors for all cases
- Increasing number of cores requires more and more electric power
 - Back to hitting the heating wall
 - Need to provide compute power with less energy
- Accelerators attached to nodes
 - More cores but specialized -> less power

Building a Supercomputer: Multiplying sockets



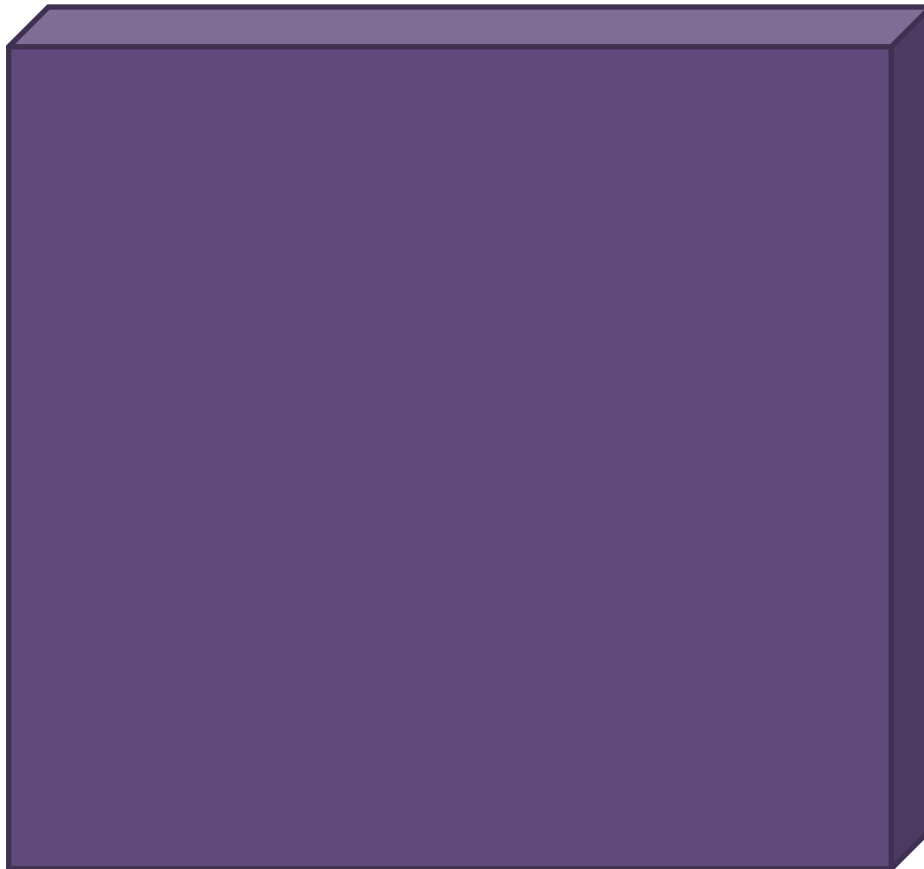
- Hard to continue extending this number of elements within a limited size
- Solution: multiply these complex structures and link them together

Building a Supercomputer: Multiplying nodes



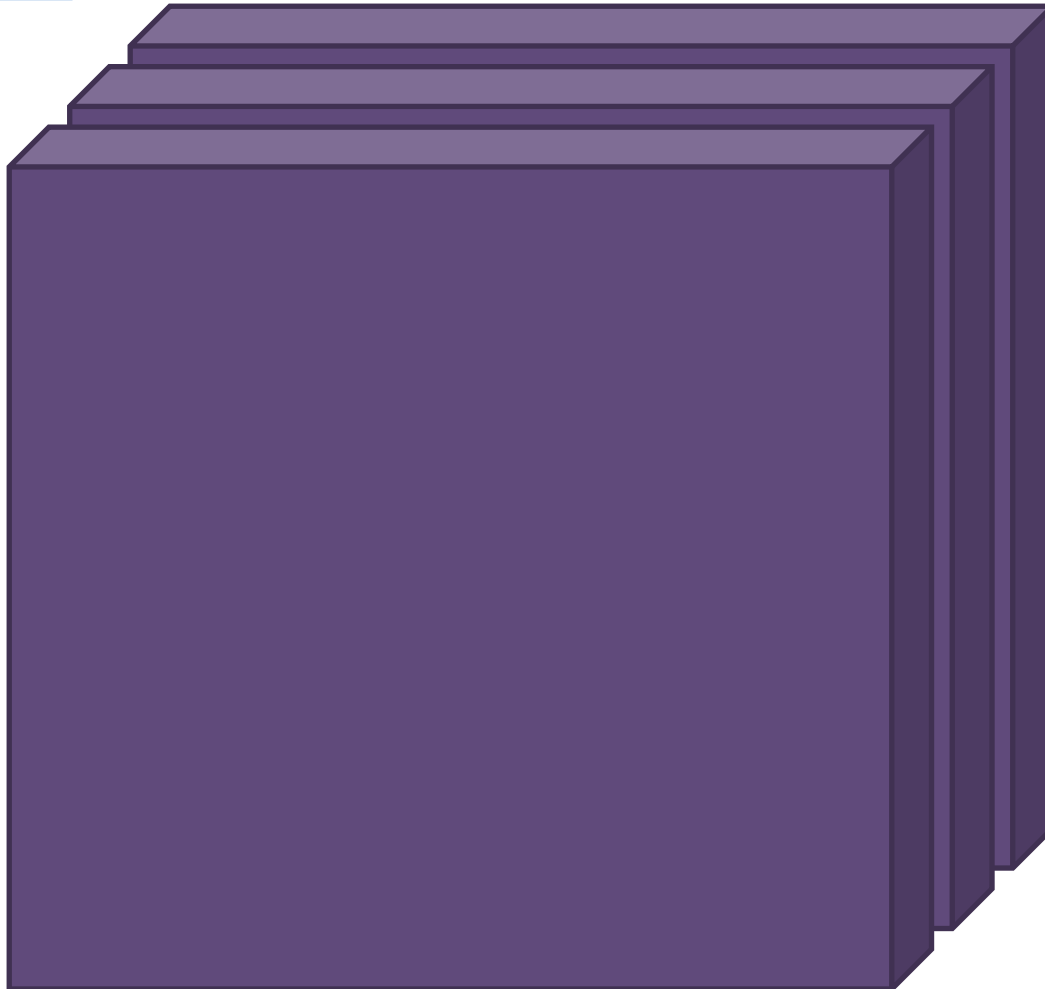
- Same, difficult to continue multiplying sockets
- Solution: multiply compute nodes, and link them together with a network

Building a Supercomputer: a cabinet



- Nodes are combined in a rack
- Racks are combined together inside a cabinet
- Need network links at every step.

Building a Supercomputer: the supercomputer



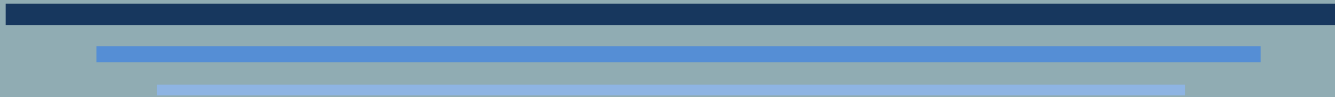
- Finally, cabinets are linked together through network links
 - Network cables
 - Network switches
- A supercomputer is the whole ensemble of cabinets
 - Compute nodes
 - I/O nodes
 - Login nodes

Example: Tera-1000



© CEA Vue d'artiste

PARALLEL COMPUTING



Parallel Computing



■ Tentative definition

- *Ability to exploit multiple compute units at the same time to solve a problem*

■ Involve various domains

- Vehicle security, performance
- Chemistry (molecule interaction/reaction)
- Bio-informatics
- Energy
- Weather forecast

Definitions



- Action
 - Work to do
- Thread
 - Implementation of an action: logical sequence of sequential operations result of the execution of a program
- Process
 - Instance of a program. A process consists of one or more threads that share a common address space. If a process has multiple threads, it is said to be multithreaded.
- Parallel computing
 - Parallel computing consists of splitting a program in several actions that can be executed at the same time on independent computing resources to reduce execution time/compute larger problems/try multiple solutions

What is Parallelism?



- Old idea to solve a problem more quickly and costly in time calculation
- One solution: to use several processing units (e.g. processors)
- Difficulty: organization of parallel actions (parallel algorithmic):
 - solve the initial problem correctly: dependencies between actions
 - the processing units must have constantly (useful) work to perform: distribution and (dynamic) load balancing

Sequential programming



- ORDERED suite of instructions to run to resolve the initial problem
- Sequential semantics: any instruction can only begin when the previous one is completed and its result available
- FIXED ORDER in the execution of all instructions
 - Regardless of actions dependencies

Parallel programming



- Several execution flows (instructions + data)
- Several instructions executed simultaneously
- Multiple processors (or cores)
- Highlights the actual dependencies between statements:
 - the task T2 depends on the task T1 iff T2 needs the result of T1 (a correct result)
 - If T2 does not depend on T1 and T1 does not depend on T2, then T1 and T2 are **independent** tasks
 - → Two independent tasks can be executed in any order, or even simultaneously (e.g. in parallel)

Dependency graph

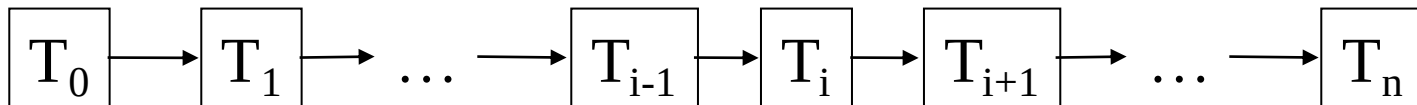
■ Dependency graph: highlights dependency relationships between tasks to complete an action

■ $T_1 \rightarrow T_2$ means that T_2 depends on T_1 ;

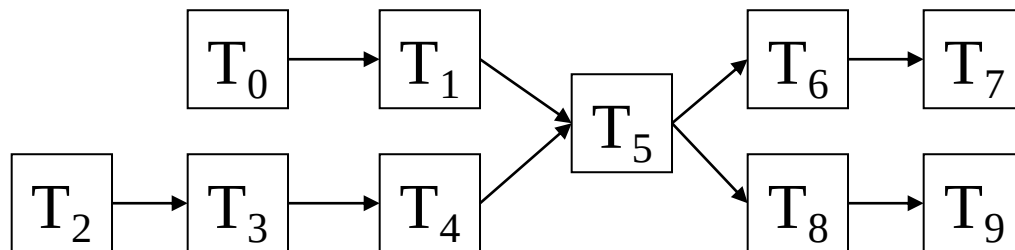
■ Depth of graph gives dependency

■ Width of graph gives independency (parallelism);

■ Sequential programming: dependency n and parallelism 1



■ Ex parallel programming with 10 tasks: dependency 6 and parallelism 2



Concurrency



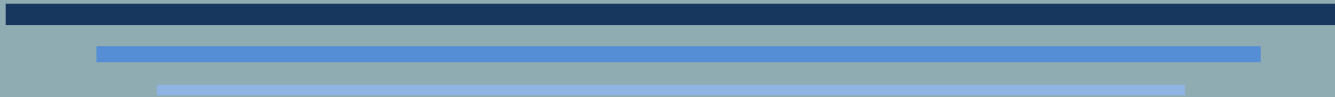
- Parallel tasks are executed:
 - **simultaneously** :
 - or **alternately** (e.g. a task can be stopped to execute another one, then its execution is resumed) ;
 - or both;
- ➔ Issue : tasks may access and modify common data (need **critical section**);
- ➔ Solution : need to find mechanisms to ensure data coherency : **locks** and **mutex** (mutual exclusion)

Communication



- Communication and synchronisation :
 - To ensure the consistency of a calculation, parallel tasks can have a « meeting point » before resuming their execution
- These « meeting points » are called **synchronisation** :
 - If the synchronisation concerns all parallel tasks
 - Global or collective synchronisation
 - If the tasks have different addressing space
 - Communications
- Communications
 - Global synchronisation => **global or collective communication**
 - synchronisation between 2 tasks => **point-to-point communication**

PARALLELISM TYPES



Parallelism types



- What are the sources of parallelism in an application?
- 3 sources :
 - Control parallelism (tasks)
 - Flow parallelism (pipeline)
 - Data parallelism

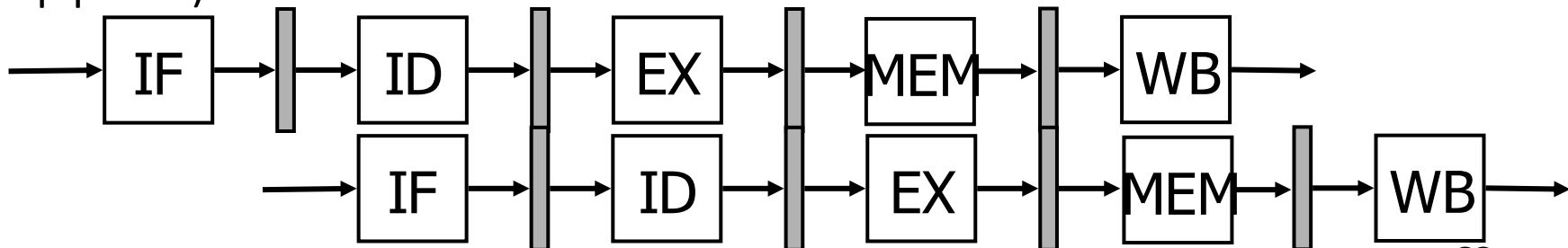
Control parallelism



- Idea: « *Do several things simultaneously* »
- Simple constatation :
 - **An application is composed of tasks that can run simultaneously**
 - Example : the execution of a kitchen recipe with several cooks
- Exploitation of control parallelism consists in managing the dependencies between an application tasks in order to obtain an allocation of computation resources as optimal as possible
 - **Extraction of this parallelism from the dependency graph: width of the graph**
- In practice, the degree of parallelism is low and often complex to set up (ex : ILP dans les processeurs)
- Fits to the following parallel programming models:
 - **MIMD (Multiple Instruction Multiple Data)**
 - **MPMD (Multiple Program Multiple Data)** : Processors run different programs with their own data

Flow parallelism

- Idea : " Chainwork "
- Work like a Pipeline
 - A series of operations are applied on a data stream, generally composed of similar data
 - The computing resources are associated with the actions and linked so that the results of the actions performed at time T are passed at time $T + 1$ to the next computing resource
 - Example : vectorial machine
- Degree of parallelism depends on pipeline depth (number of stages)
- Working on vectorial data:
 - Vectors must be long enough to minimise on the cost of filling the pipeline
- Data flow must be as contiguous as possible (every stop empties the pipeline)



Data parallelism



- Idea : « *Repeat actions on similar data* »
- Share the data and not the tasks
 - example : several kitchen clerks peeling a bunch of potatoes
- Degree of parallelism potentially very high, depending on the amount of data
- Fits the parallel programming model: **SPMD (Single Program Multiple Data)**
 - All processors run the same program with their own data
 - Viable for a large number of data
 - Very effective for many intensive computing algorithms (scientific computation, animated films): afterwards, we will focus on this programming model

Parallel programming paradigm



- Independently of the hardware architectures of the machines, two parallel programming models emerge:
 - distributed memory programming model
 - shared memory programming model
- In theory, each model can be implemented on any type of architecture with collateral effects of varying performance

distributed memory prog. model



- Conditions :
 - the parallel tasks work on separate memories, invisible from one another
 - the data (tables, etc ...) are split (called distributed data) on the various parallel tasks
- Consequence: to ensure the accuracy of the final result, inter-task communication becomes mandatory
- This is called programming by **message passing**

distributed memory prog. model



- Implementation on distributed-memory architecture
 - Easy because it is the same principle
 - on a distributed memory machine, processes that have their own address spaces must send messages over the network to exchange information
- Implementation on shared-memory architecture
 - Is not more difficult:
 - through multiple processes using the shared memory segments for communications
 - Through a multithreaded process using its memory to exchange information between threads

distributed memory prog. model



- Encapsulation of message exchanges, implemented by libraries, such as:
 - PVM (Parallel Virtual Machine) : one of the first portable library of message passing
 - MPI (Message Passing Interface) : the actual de-facto standard, coming from the collaborations of industrial partners and universities
 - This cours is dedicated to this programming interface
 - RMA: Remote Memory Access

shared memory prog. model



- Condition :
 - **Parallel tasks are sharing the same address space**
- Consequence :
 - Need to properly handle concurrent access to the same data (critical sections)
- Despite the apparent simplicity of programming, performance can be quickly degraded because:
 - Critical sections are not parallel (by definition)
 - Data locality and memory hierarchy are transparent to the user



shared memory prog. model



- Implementation on distributed-memory architecture
 - difficult : how to « see » the whole memory ?
 - DSM (Distributed Shared Memory) : software mechanism allowing to simulate a unified memory on top of a physically distributed memory
 - May be very inefficient because the cost of distant accesses is hidden
- Implementation on shared-memory architecture
 - Easy because it is the same principle
 - In a multithreaded process, every thread can access the process memory (which can address the whole memory on the node)

shared memory prog. model



- API POSIX pthread :
 - Normalised manipulation (POSIX) of threads in a process
 - Fit the MPMD model
- OpenMP :
 - Thread manipulation by directives
- At the present time, tools implementing the shared memory model offer a great deal of thought so that applications can make the best use of multicore processors
 - TBB, Cilk++, ABB, ...

Hybrid memory prog. model



- Combine both distributed and shared memory programming models
- Explicit hybrid programming:
 - Explicitly use multiple programming models for distributed and shared memory paradigm
 - MPI+X (X= OpenMP, Pthreads, TBB, ...)
- Implicit programming model
 - Programming models working on a unified view of the memory
 - PGAS: Partitioned Global Address Space
 - Software to simulate a unified memory
 - DSM: Distributed Shared Memory

Heterogeneous prog. model



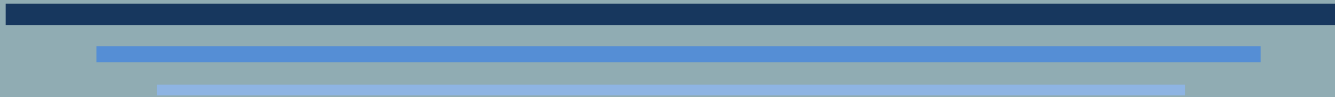
- **Programming on accelerator(s) attached to the node**
 - GPUs, Intel Knights Corner/Landing, Specialized cards
- **Specific programming language**
 - Ex: CUDA for Nvidia GPUs, Intel LEO for KNL
- **Can also use generic programming models**
 - C, C++, etc... for Intel KNL
 - OpenMP accelerator directives for Intel KNL, Nvidia...

Parallelism benefit in cluster



- Distributed memory programming model
 - Allows to work on all nodes of a supercomputer
 - Parallelism potential benefits: tens (or hundreds) of thousands nodes
- Shared memory programming model
 - Allows to work on all cores/hyperthreads of a node
 - Parallelism potential benefits: up to 288 hyperthreads on an Intel KNL
- Vectorization
 - Allows to use all computational resources in one instruction
 - Parallelism potential benefits (depends on architecture vector length): up to 16 doubles / 32 simples

DISTRIBUTED PARALLEL PROGRAMMING



Message Exchange

Message Exchange



- Message characteristics
 - Sender
 - Destination task
 - Data to exchange
- High-level protocol
 - Pair of actions will resolve message exchange
 - Sender must send the message
 - Let's consider a function called *send*
 - Recipient must receive the message
 - Let's consider a function called *recv*

Main Principle

- Two parallel tasks **T0** et **T1**
 - Distinct memory space
 - Each task has its own instructions to execute

T0 Task

```
instruction1;  
instruction2;
```

T1 Task

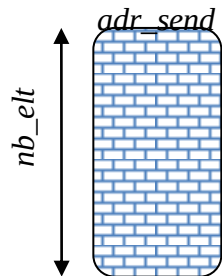
```
instruction1;  
instruction2;
```

Main Principle

- **T1** depends on **T0**
 - **T0** must send data to **T1**
 - Data are located in *adr_send* with *nb_elt* elements

T0 Task

```
instruction1;  
instruction2;  
send(adr_send, nb_elt, T1);
```



T1 Task

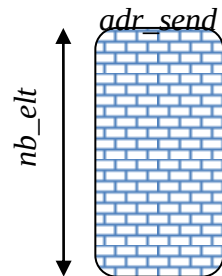
```
instruction1;  
instruction2;
```


Main Principle

- **T1** must receive data from **T0** (*recv*)
 - Size of message *nb_elt* should be known by recipient
 - Recipient may have to allocate a memory zone to get the received data (zone pointed by *adr_recv*)

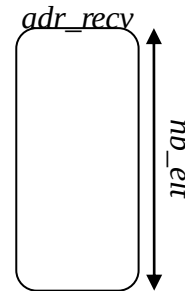
T0 Task

```
instruction1;  
instruction2;  
send(adr_send, nb_elt, T1);
```



T1 Task

```
instruction1;  
instruction2;  
recv(adr_recv, nb_elt, T0);
```

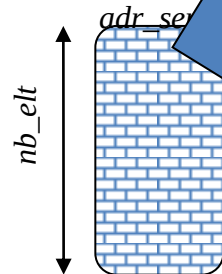


Main Principle

- Communication
 - *send* blocks **T0** until data are sent
 - *recv* blocks **T1** until data are received

T0 Task

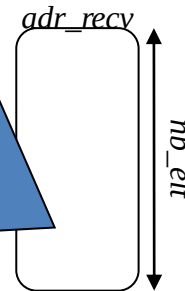
```
instruction1;  
instruction2;  
send(adr_send, nb_elt, T1);
```



Data Transfer

T1 Task

```
instruction1;  
instruction2;  
recv(adr_recv, nb_elt, T0);
```

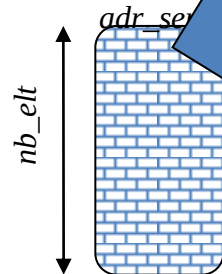


Main Principle

- Communication
 - *send* blocks **T0** until data are sent
 - *recv* blocks **T1** until data are received

T0 Task

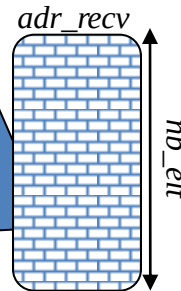
```
instruction1;  
instruction2;  
send(adr_send, nb_elt, T1);
```



Data Transfer

T1 Task

```
instruction1;  
instruction2;  
recv(adr_recv, nb_elt, T0);
```

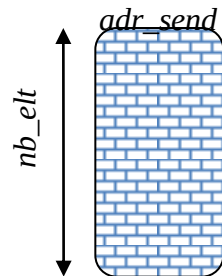


Main Principle

- **T1** owns a complete copy of data sent by **T0**

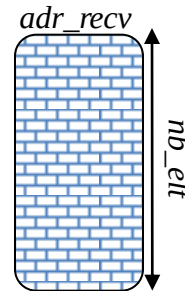
T0 Task

```
instruction1;  
instruction2;  
send(adr_send, nb_elt, T1);
```



T1 Task

```
instruction1;  
instruction2;  
recv(adr_recv, nb_elt, T0);
```

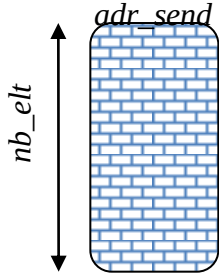


Main Principle

- Tasks **T0** and **T1** may continue their execution
- Following instructions of **T1** may access to data stored at address *adr_rcv*

T0 Task

```
instruction1;  
instruction2;  
send(adr_send, nb_elt, T1);
```

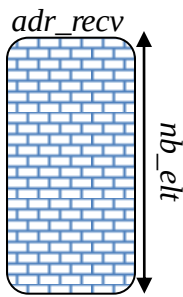


The diagram shows a vertical rectangular buffer with a blue brick-like pattern. A double-headed vertical arrow to the left of the buffer is labeled *nb_elt*. The top of the buffer is labeled *adr_send*.

```
instruction3;
```

T1 Task

```
instruction1;  
instruction2;  
recv(adr_rcv, nb_elt, T0);
```



The diagram shows a vertical rectangular buffer with a blue brick-like pattern. A double-headed vertical arrow to the right of the buffer is labeled *nb_elt*. The top of the buffer is labeled *adr_rcv*.

```
instruction3;
```

Example



- Parallel sum on each element of an array
- Hypothesis
 - Array t with N floats (N is even)
 - Array t is distributed across 2 tasks T_0 and T_1
 - Parallelism type: data
- Goal
 - T_1 must print the sum of each element of t
- Code?

Example



T0 sends its partial sum to T1

T0

```
double p = 0.0;  
int i;
```

```
for( i=0 ; i<N/2 ; i++ )  
    p += tab[i];
```

```
send(&p, 1, T1);
```

T1 needs partial sum from T0

T1

```
double p = 0.0;  
double s;  
int i;
```

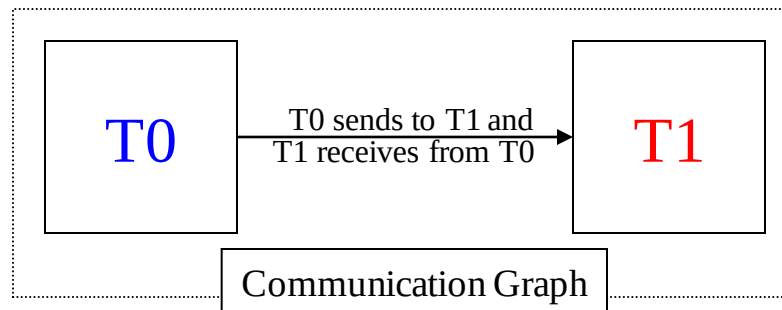
```
for( i=0 ; i<N/2 ; i++ )  
    p += tab[i];
```

```
recv(&s, 1, T0);
```

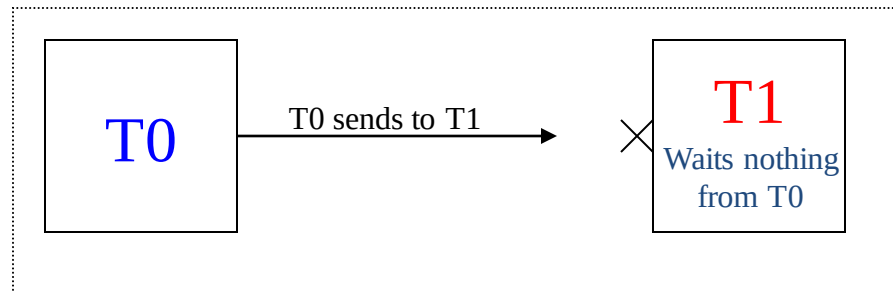
```
printf("%g",s+p);
```

Send/Recv Matching

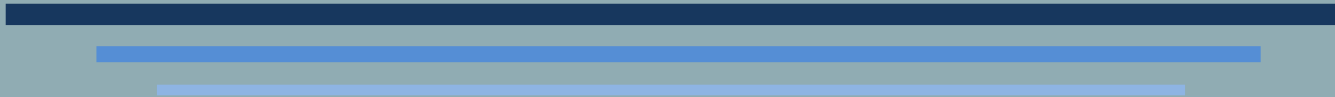
- Every *send* corresponds to one *recv* (and vice-versa)
- Model with an oriented graph
 - Vertices are tasks
 - Edges are communications



- A missing send or receive action lead to a deadlock situation



INTRODUCTION TO MPI



Introduction



- MPI: Message-Passing Interface
- High-level API (Application Programming Interface)
 - Parallel programming
 - Distributed-memory paradigm
- Implementation as a library
 - Interface through functions
- Language compatibility
 - C
 - C++
 - FORTRAN

Why use MPI ?

- MPI is mostly an interface
- MPI is available in every type of parallel architectures
- MPI supports heavy parallelism
- Machine and/or network vendors often provides their own optimized version of MPI library
- MPI is also available in *open source* for most of current supercomputer architectures
 - MPICH2 : <http://www.mcs.anl.gov/research/projects/mpich2/>
 - OpenMPI : <http://www.open-mpi.org>

Hello World!

```
#include <stdio.h>
/* MPI function signatures */
#include <mpi.h>

int main(int argc, char **argv)
{
    /* Initialization of MPI */
    MPI_Init(&argc, &argv);

    printf("Hello World!\n");

    /* Finalization of MPI */
    MPI_Finalize();
    return 0;
}
```

- Header file
 - Need to include it
 - Contains signatures of each available MPI function
 - Function bodies are located inside a library
- Syntax
 - All functions related to MPI start with MPI_
- Convention
 - No MPI calls before MPI_Init
 - No MPI calls after MPI_Finalize

Compilation

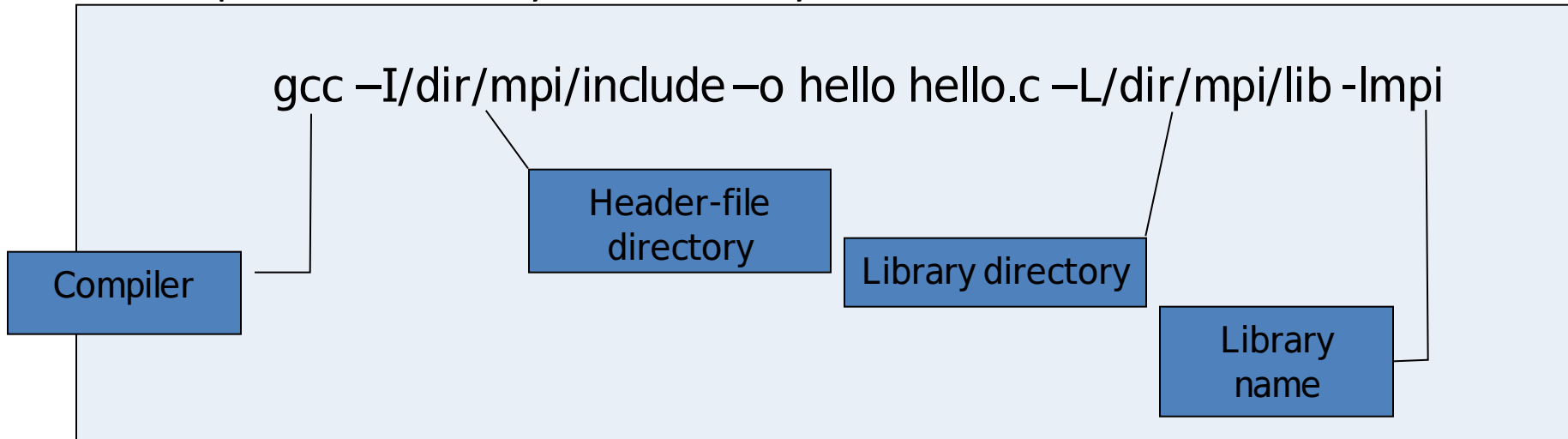


- Basically
 - *Compilation process like any other library*
- But multiple ways to compile an MPI program
 - Simple way: rely on `mpicc` script
 - Complex way: launch regular compiler with options to specify paths to the library
- Simple way
 - Script/program that hide the library configuration details

```
mpicc -o hello hello.c
```
 - Call the default underlying compiler
 - Possible to change the compiler that will be invoked

Compilation

- Complex way
 - Without the script → pass right options for library configuration
- Generic mandatory options to use external library
 - Directory where header files are located (e.g., `mpi.h`)
 - Directory where library files are located (e.g., `libmpi.so`)
 - Name of the library to use (*linker*)
- Example: `libc` library or MPI library



Execution w/ Job Manager

- Slurm can spawn MPI processes
 - Rely on `srun` command
- If not available
 - Use of `mpirun` script (different syntax and usage)

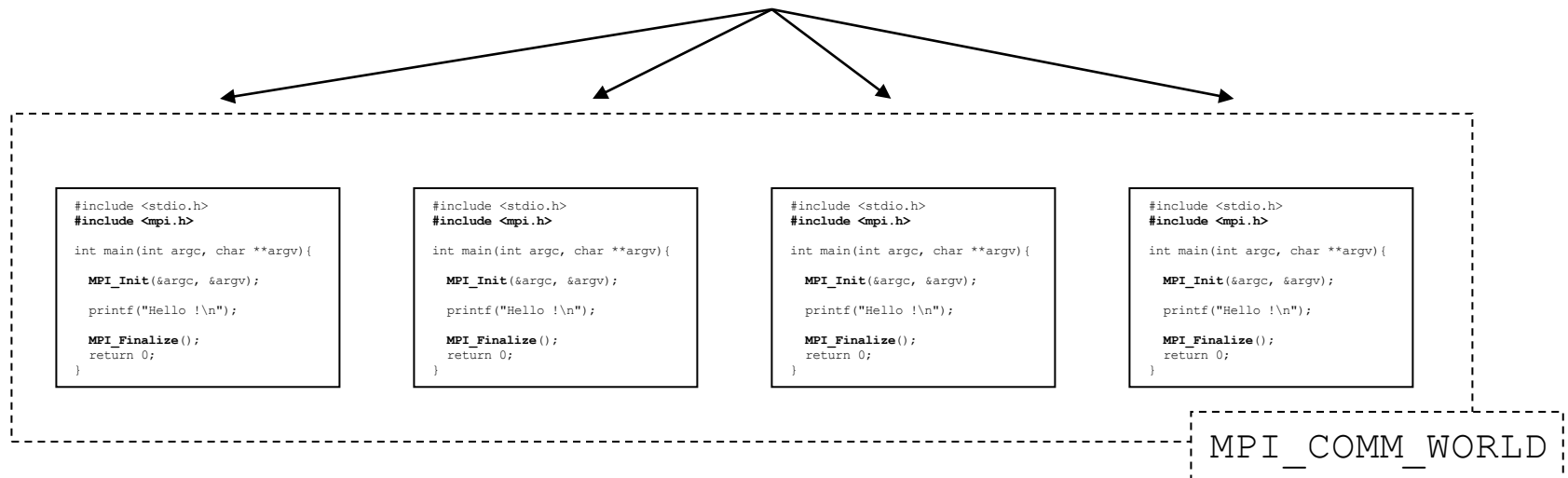
```
srun -n 4 ./hello
```

```
Hello World!  
Hello World!  
Hello World!  
Hello World!
```

- Remarks
 - Creation of 4 processes
 - Every process has the same instructions
 - Processes are independent for execution

Communicator

`srun -n 4 ./hello`



- Group of processes form a communicator
 - Predefined: `MPI_COMM_WORLD` w/ all processes
- Communicator = set of processes + communication context
 - Type: `MPI_Comm`

Total Number of Processes

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv) {
    int N;
    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &N);

    printf("Number of processes = %d\n", N);

    MPI_Finalize();
    return 0;
}
```

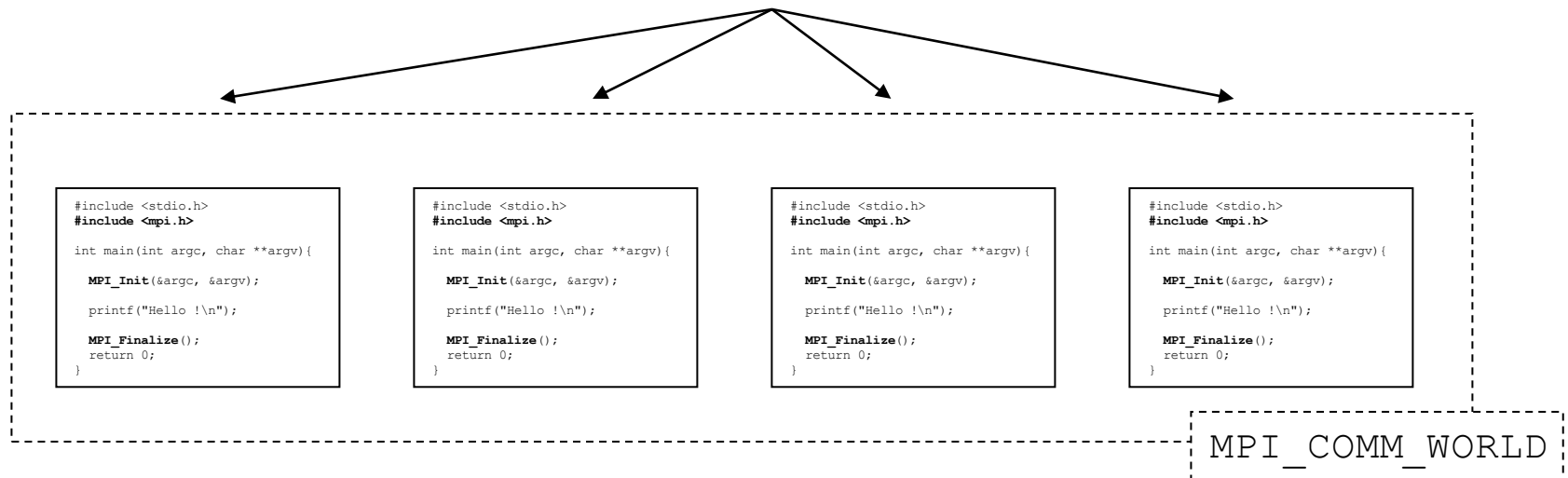
```
% srun -n 4 ./a.out
Number of processes = 4
Number of processes = 4
Number of processes = 4
Number of processes = 4
%
```

```
int MPI_Comm_size( MPI_Comm comm, int *size);
```

- Return size of communicator `comm` in `*size`
- If `comm == MPI_COMM_WORLD`, `MPI_Comm_size` returns the total number of MPI processes in the application

Process Rank

`srun -n 4 ./hello`



- Inside a communicator, MPI assigns rank from 0 to size-1
 - This is the rank of a process
- Function `MPI_Comm_rank` returns the rank in the communicator `comm` inside the address `*rank`:

```
int MPI_Comm_rank(MPI_Comm comm, int *rank);
```

Process Rank



```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv) {
    int N, me;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &N);
    MPI_Comm_rank(MPI_COMM_WORLD, &me);
    printf("My rank is %d out of %d\n", me, N);
    MPI_Finalize();
    return 0;
}
```

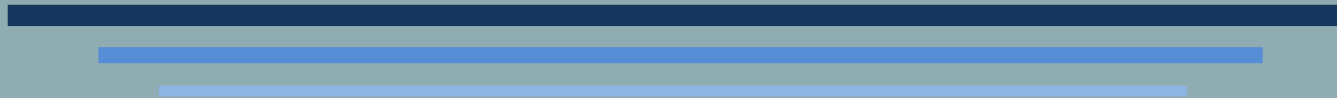
```
% srun -n 4 ./a.out
My rank is 1 out of 4
My rank is 0 out of 4
My rank is 3 out of 4
My rank is 2 out of 4
%
```

Process Rank



- Number of processes may differ from number of available cores/processors!
 - By default, Slurm binds one MPI process to one core
 - Option `-c` can be used to book multiple cores per rank
- Execution of processes is not related to their rank
 - Parallel execution
 - At the beginning, no ordering between processes
 - Only communications can imply some partial ordering
- Rank is usually used to determine
 - Which part of data should I work on?
 - What is my role (master/worker)?

MPI POINT-TO-POINT COMMUNICATIONS



Send/Recv

MPI Communication



- MPI is a parallel distributed-memory model
 - Each process accesses its own memory space
 - Based on message passing
- What is the main interface for data exchange w/ MPI?
- To send a message
 - `MPI_Send` function

Sending Messages

- Function to send a message

```
int MPI_Send (  
    void *buf(in),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int dest(in),  
    int tag(in),  
    MPI_Comm comm(in)  
);
```

Main characteristics of message to send

Sending Messages

- Function to send a message

```
int MPI_Send (  
    void *buf(in),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int dest(in),  
    int tag(in),  
    MPI_Comm comm(in)  
);
```

Data
address

Data to send inside an
array pointed by **buf**
whose elements are of
type **datatype**.

MPI predefined scalar
types corresponding to
existing C types.

Sending Messages

<i>MPI_Datatype</i>	<i>C Type</i>
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	<i>One byte</i>
MPI_PACKED	<i>Pack of non-contiguous data</i>

Sending Messages



```
int MPI_Send (  
    void *buf(in),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int dest(in),  
    int tag(in),  
    MPI_Comm comm(in)  
);
```

Message size is **count**.

Not in bytes, but in
number of elements of
type **datatype**
(portable way to
express size).

Sending Messages



```
int MPI_Send (  
    void *buf(in),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int dest(in),  
    int tag(in),  
    MPI_Comm comm(in)  
);
```

Communicator for message.

Communicator = (sub-)set
of processes +
communication context

MPI_COMM_WORLD
contains all processes
created during application
launch

Sending Messages



```
int MPI_Send (  
    void *buf(in),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int dest(in),  
    int tag(in),  
    MPI_Comm comm(in)  
);
```

Recipient rank.

This rank is valid inside
communicator **comm**.

For
MPI_COMM_WORLD,
dest should be between
0 and number of ranks
(excl.).

Sending Messages



```
int MPI_Send (  
    void *buf(in),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int dest(in),  
    int tag(in),  
    MPI_Comm comm(in)  
);
```

Label named **tag**
used to identify
messages.

Allows distinguish
messages from the
same sender and the
same recipient.

Remarks on Sending Messages



- `MPI_Send` is blocking function
 - Returning from `MPI_Send`, process can manipulate (e.g., write) the data buffer containing the message
- How to determine the message tag
 - Can use any way you want
 - Not necessary for different send/recipient pair
 - Example:
$$\text{tag} = \text{src} * N + \text{dest}$$
(N total number of MPI processes, `src` sender rank, `dest` recipient rank;)
- Be careful: the number of tags is limited!

MPI Communication



- What is the main interface for data exchange w/ MPI?
- Message reception
 - `MPI_Recv` function

Receiving Messages



```
int MPI_Recv (  
    void *buf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int source(in),  
    int tag(in),  
    MPI_Comm comm(in),  
    MPI_Status *status(out)  
);
```

} Main characteristics of message to receive

Receiving Messages

```
int MPI_Recv (  
    void *buf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int source(in),  
    int tag(in),  
    MPI_Comm comm(in),  
    MPI_Status *status(out)  
);
```

Address of memory zone to put the received data.

This zone should be allocated in some way BEFORE!

Max size of received message

Unit is in number of elements of type **datatype**.

The actual size of received message is less or equal to **count**.

Receiving Messages

```
int MPI_Recv (  
    void *buf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int source(in),  
    int tag(in),  
    MPI_Comm comm(in),  
    MPI_Status *status(out)  
);
```

Rank of sender.

Rank should be valid in
comm communicator.

Can specify the predefined
value `MPI_ANY_SOURCE` →
May match a message from
any sender in the target
communicator

Message tag.

Should be the same of the
one put in corresponding
`MPI_Send` function call.

Receiving Messages



```
int MPI_Recv (  
    void *buf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int source(in),  
    int tag(in),  
    MPI_Comm comm(in),  
    MPI_Status *status(out)  
);
```

Information about received message

Information and Status

- MPI_Status is a C structure

```
struct MPI_Status{  
    int MPI_SOURCE; /* message sender (useful w/ MPI_ANY_SOURCE argument) */  
    int MPI_TAG; /* message tag (useful w/ MPI_ANY_TAG argument) */  
    int MPI_ERROR; /* error code */  
};
```

- If message size is unknown to the recipient, it is possible to extract the actual size with MPI_Get_count

```
int MPI_Get_count(  
    MPI_Status *status(in), /* status returned by MPI_Recv */  
    MPI_Datatype datatype(in), /* Type of elements in the message */  
    int *count(out) /* Size of the message (in number of elements of type datatype) */  
);
```

Simple MPI Example



```
int main(int argc, char **argv) {
    double p = 0., s0;
    int i, r;
    MPI_Status status;

    MPI_Init(&argc, &argv); /* Initialization of MPI library */
    MPI_Comm_rank(MPI_COMM_WORLD, &r); /* Get the rank of current rank */

    for( i = 0 ; i < N/2 ; i++ )
        p += tab[i];

    tag = 1000; /* Message tag */
    if (r == 0) {
        MPI_Send(&p, 1, MPI_DOUBLE, 1, tag, MPI_COMM_WORLD);
    } else {
        MPI_Recv(&s0, 1, MPI_DOUBLE, 0, tag, MPI_COMM_WORLD, &status);
        printf( "Sum = %d\n", s0+p );
    }
    MPI_Finalize();
    return 0;
}
```

Simple MPI Example

```
sum = 0.; /* Each process has N/P elements of distributed */
for( i = 0 ; i < N/P ; i++ ) /* array and perform a partial sum */
    sum += tab[i];

if (r == 0) {
    /* Process 0 receives P-1 messages in any order */
    for( t = 1 ; t < P ; t++ ) {

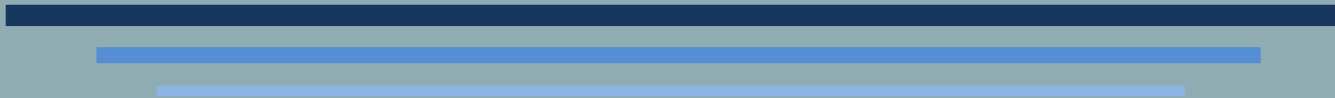
        MPI_Recv(&s, 1, MPI_DOUBLE,
                 MPI_ANY_SOURCE, MPI_ANY_TAG, /* wildcards */
                 MPI_COMM_WORLD, &sta);

        printf(« Message from rank %d\n", sta.MPI_SOURCE);

        sum += s; /* Contribution of process sta.MPI_SOURCE to the global sum */
    }
} else {

    /* Other processes send their partial sum to rank 0 */
    MPI_Send(&som, 1, MPI_DOUBLE, 0, rang, MPI_COMM_WORLD);
}
```

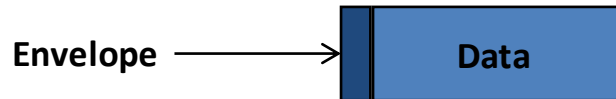
MESSAGE PASSING PROTOCOLS



Eager, Rendez-vous, Short

Message protocols

- Message consists of “envelope” and data
 - Envelope contains tag, communicator, length, source information, plus impl. private data



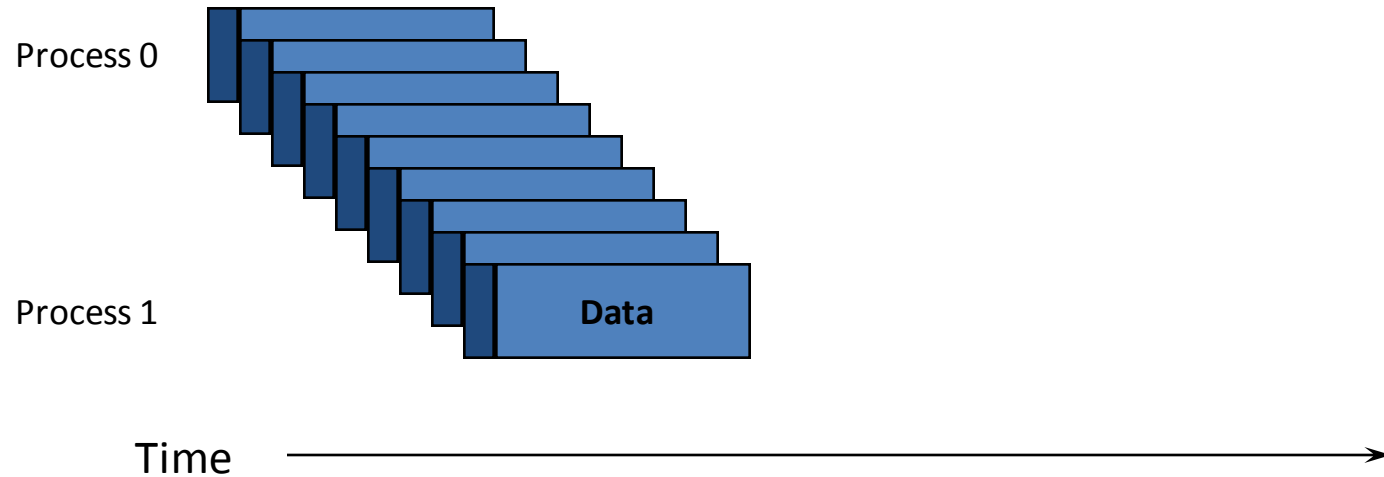
- Short
 - Message data (message for short) sent with envelope
- Eager
 - Message sent assuming destination can store
- Rendezvous
 - Message not sent until destination oks

Message Protocol Details



- User versus system buffer space
- Packetization
- Collective operations
- Datatypes, particularly non-contiguous
 - Handling of important special cases
 - Constant stride
 - Contiguous structures

Eager Protocol



- Data delivered to process 1
 - No matching receive may exist; process 1 must then buffer and copy.

Eager Features



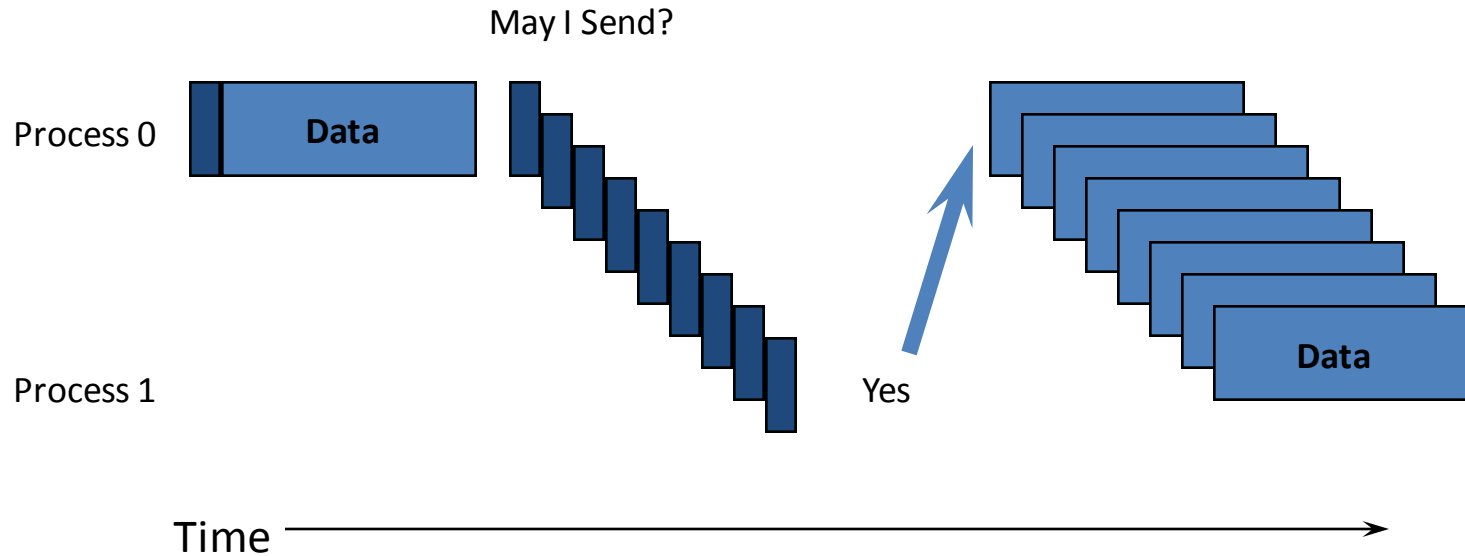
- Reduces synchronization delays
- Simplifies programming (just MPI_Send)
- Requires significant buffering
- May require active involvement of CPU to drain network at receiver's end
- May introduce additional copy (buffer to final destination)

How Scalable is Eager Delivery?



- Buffering must be reserved for arbitrary senders
- User-model mismatch (often expect buffering allocated entirely to “used” connections).
- Common approach in implementations is to provide same buffering for all members of `MPI_COMM_WORLD`; this is optimizing for non-scalable computations
- Scalable implementations that exploit message patterns are possible

Rendezvous Protocol



- Envelope delivered first
- Data delivered when user-buffer available
 - Only buffering of envelopes required

Rendezvous Features



- Robust and safe
 - (except for limit on the number of envelopes...)
- May remove copy (user to user direct)
- More complex programming (waits/tests)
- May introduce synchronization delays (waiting for receiver to ok send)

Short Protocol



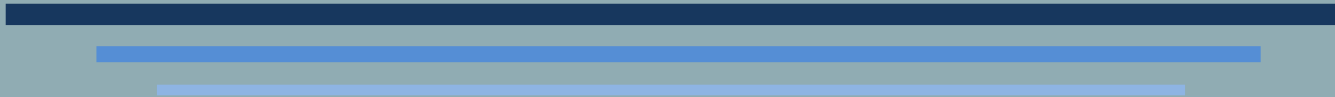
- Data is part of the envelope
- Otherwise like eager protocol
- May be performance optimization in interconnection system for short messages, particularly for networks that send fixed-length packets (or cache lines)

Special Protocols for DSM



- Message passing is a good way to use distributed shared memory (DSM) machines because it provides a way to express memory locality.
- Put
 - Sender puts to destination memory (user or MPI buffer). Like Eager.
- Get
 - Receiver gets data from sender or MPI buffer. Like Rendezvous.
- Short, long, rendezvous versions of these

MPI POINT-TO-POINT COMMUNICATIONS



Blocking, Bufferized, Ready, Non-Blocking

Blocking Communications



- Definition of operations
 - An **operation** is a set of one or more procedures leading from a well-defined input state to a well_defined output state. An operation consists in four stages: Initialization, starting, completion and freeing.
 - **Initialization** hands over the argument list to the operation but not to the content of the message buffers. For an operation, it may be specified that array arguments must not be changed until the operation is freed

Blocking Communications



- Definition of operations
 - **Starting** hands over the content of the message data buffer to the associated operation.
 - Note that **initiation** is an abbreviation for the combination of the initialization and starting stages.
 - **Completion** returns control of the content of the message data buffers and indicates that any output buffers have been updated
 - **Freeing** returns control of the rest of the argument list

Blocking Communications



■ Definition

- A **procedure is blocking** if return from the procedure indicates the user is allowed to reuse resources specified in the call.
- For a **blocking operation**, all four stages are combined in a single blocking procedure call.

Blocking Communications



■ Definition

- A *send* is **blocking** if after performing *send* it is possible to manipulate (read/write) the input data buffer without corrupting the communication

■ Meaning

- A blocking *send* will not return while the communication library is not able to handle the message
- It doesn't mean that
 - Message has been received
 - Message has been sent !!!

Blocking Communications



- After *send*, **T0** may modify the value of *a*
- **T1** will receive 100 (value of *a* as input of *send* by **T0**)
- Note
 - Resolving a blocking send does not mean that the receiver has the message



Blocking Communications



- Definition

- A *recv* is **blocking** if after performing *recv* the output buffer contains the received message

- Meaning

- A blocking *recv* will not return while the message has not been received and processed

Blocking Communications



- After *send*,
 - **T0** may manipulate *a* and its content
- After *recv*,
 - Content of output buffer (*a* in **T1**) can be manipulated (read, write, print...) without concurrency issue

Blocking Communications



- `MPI_Send` et `MPI_Recv` are blocking
 - `MPI_Send` returns when data buffer can be manipulate again by sender
 - `MPI_Recv` returns when the message arrived and has been processed
- Issue?
 - Be careful to deadlock situations!

Ring Topology

```
left = (rank + P - 1) % P;
```

```
right = (rank + 1) % P;
```

```
if (rank == 0)
```

```
    m = 0;
```

```
/* Receiving from left-hand side */
```

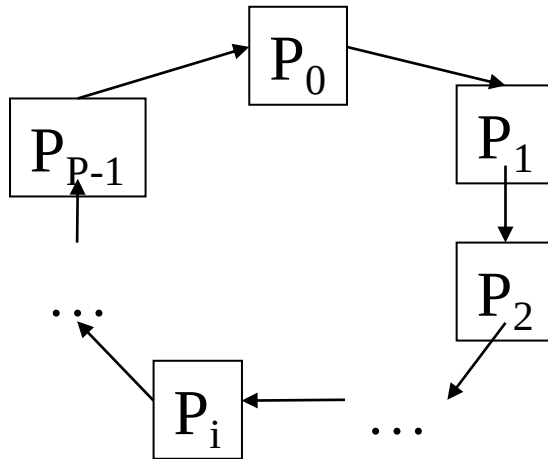
```
MPI_Recv(&m, 1, MPI_INT, left, tag1, MPI_COMM_WORLD, &sta);
```

```
/* Sending to right-hand side */
```

```
MPI_Send(&m, 1, MPI_INT, right, tag2, MPI_COMM_WORLD);
```

Ring example:

Processes pass one message in increasing rank value

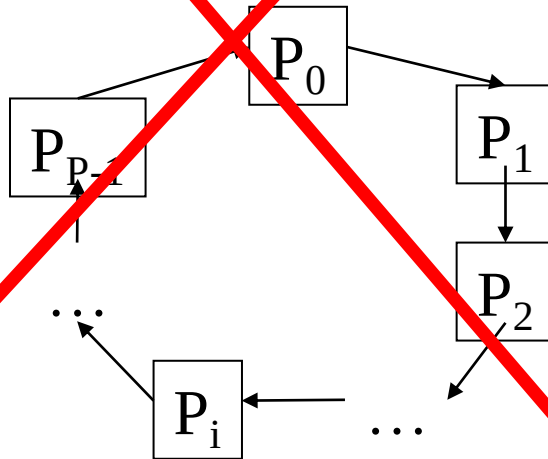


Ring Topology

```
left = (rank + P - 1) % P;  
right = (rank + 1) % P;  
  
if (rank == 0)  
    m = 0;  
  
/* Receiving from left-hand side */  
MPI_Recv(&m, 1, MPI_INT, left, tag1, MPI_COMM_WORLD, &sta);  
  
/* Sending to right-hand side */  
MPI_Send(&m, 1, MPI_INT, right, tag2, MPI_COMM_WORLD);
```

Ring example:

Processes pass one message in increasing rank value



Each process P_i waits a message from P_{i-1} before sending it to P_{i+1} . To do so, P_{i-1} should send this message, but P_{i-1} is blocked because it wait for a message from P_{i-2} ...

⇒ **deadlock**

Communication Mode



- Multiple modes for blocking communications
 1. **Synchronous mode**
 2. Buffered mode
 3. Standard mode

Synchronous Mode



- Definition

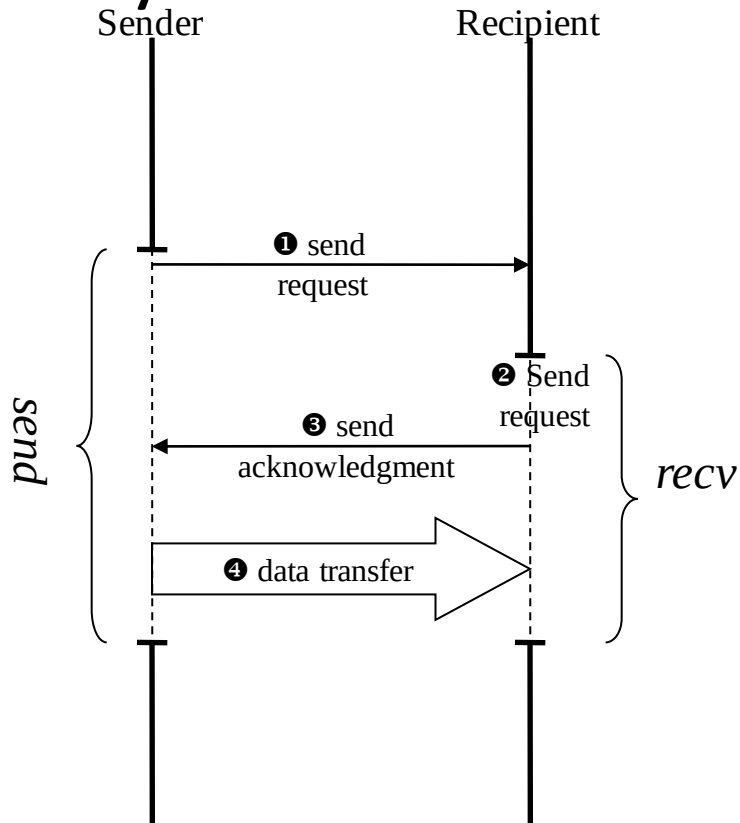
- A **synchronous send** will block while the message has not been received by the recipient

- Implementation

- Require some sort of synchronization mechanism between sender and recipient
- Design of a data-transfer protocol

Synchronous Mode

■ Synchronous communication protocol



- ❶ For a synchronous send, sender transfer a request to the receiver and waits for an answer
- ❷ When recipient starts the `recv` function, it waits for a sender request
- ❸ When recipient has the expected request, it answers with an acknowledgment message
- ❹ Sender and recipient are now synchronized leading to a safe data transfer

Synchronous Mode



■ Advantages

- No intermediate copy inside internal buffer
- May rely on optimized direct remote memory access (DMA or RDMA)

■ Drawbacks

- Involve a remote synchronization (like *rendez-vous*) between the two MPI processes
- May lead to idle overhead

■ Optimal situation

- When sender and recipient calls the corresponding function *at the same time*
- Possible in data parallelism when load is balanced between the two MPI processes

MPI Synchronous Mode



```
int MPI_Ssend (  
void *buf(in),  
int count(in),  
MPI_Datatype datatype(in),  
int dest(in),  
int tag(in),  
MPI_Comm comm(in)  
);
```

- Same signature as MPI_Send.
- Receive with function MPI_Recv



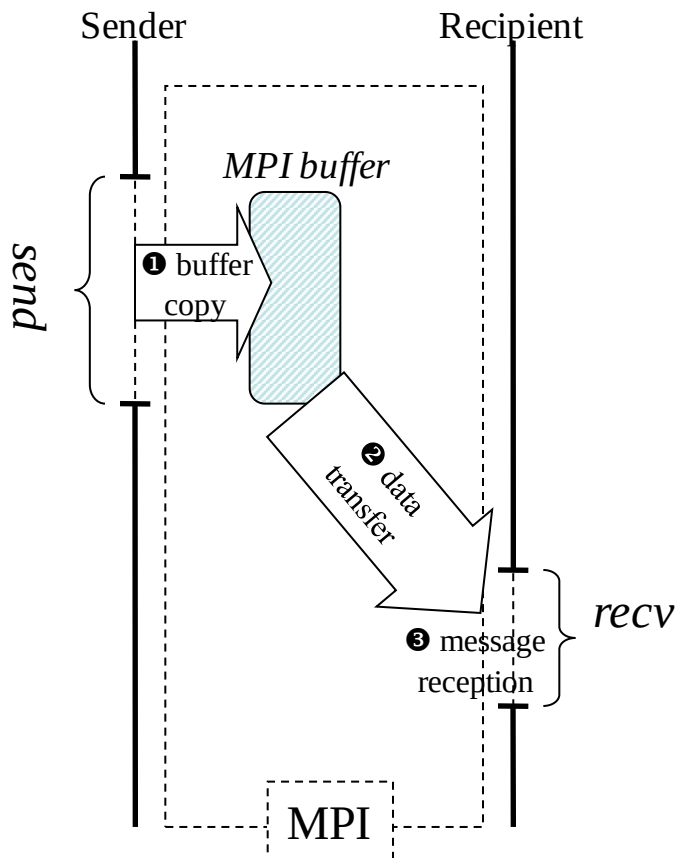
Communication Mode



- Multiple modes for blocking communications
 1. Synchronous mode
 2. **Buffered mode**
 3. Standard mode

Buffered Mode

- Waits until message has been copied to internal buffer
- Protocol:



- ➊ Sender copies incoming message inside a buffer (managed by the communication library). Send function may return
- ➋ Communication library owns a copy of the data to transfer and sends it to the recipient
- ➌ Recipient gets the message asap

Buffered Mode



■ Advantages

- Ability to decouple *send* and *recv* actions: *send* may return before recipient calls *recv* function

■ Drawbacks

- Intermediate data copy
 - CPU overhead
 - Memory consumption overhead
 - Memory bandwidth overhead
- Limited to an upper bound (buffer size)

■ Optimal situations

- When *send* and *recv* functions are not posted *at the same time*
- Load is not balanced between MPI processes

Buffer Allocation

- User may provide its own buffer to replace the internal one.
 - Function to attach user-allocated buffer `buf` of size `sz` bytes

```
int MPI_Buffer_attach(void *buf, int sz);
```
- Such buffer can be released and used again in the application by the user
 - Function to detach a user-allocated buffer
 - Return the buffer start address and its size

```
int MPI_Buffer_detach(void **buf_adr, int *sz);
```

Buffer Allocation

```
#define BUFFSIZE 100000
int sz;
char *buf;

MPI_Buffer_attach( malloc(BUFFSIZE), BUFFSIZE );
...
MPI_Bsend(msg1, ...);
MPI_Bsend(msg2, ...);
...
MPI_Buffer_detach( &buf, &sz );
free(buf);
```

- Only used in MPI_Bsend
- Only one buffer may be attached
- Only useful for sender



Communication Mode



- Multiple modes for blocking communications
 1. Synchronous mode
 2. Buffered mode
 3. **Standard mode**

Standard Mode



- Function for standard communication
 - `MPI_Send`
- Standard communication protocol
 - MPI includes an internal threshold T
 - If input message size is lower than T
 - Switch to buffered mode
 - If input message size is larger than T
 - Switch to synchronous mode

Standard Mode

```
if ( rang == 0 )
voisin = 1;
else if ( rang == 1 )
voisin = 0;

MPI_Send(&msg1, N, MPI_BYTE, voisin, tag1, comm);
MPI_Recv(&msg2, N, MPI_BYTE, voisin, tag2, comm);
```

- Is this code safe?
- NO
 - If N is small enough → OK
 - If N is too large → Deadlock

Standard Mode



- Hint to detect such issues
 - Replace calls to `MPI_Send` by `MPI_Ssend`
 - Whatever the size of messages and scheduling, the applications should not deadlock
- Deadlocks means application bug!